llmsays: One-Line LLM Inference with Automatic Prompt-Tier Routing and Multi-Provider Failover

Abhiraj Adhikary[1], Anik Chand[2], and Rudra Prasad Bhowmick[3]

[1]Department of Data Science, Haldia Institute of Technology, India.
abhirajadhikary06@gmail.com
[2]Department of Computer Science and Engineering, Haldia Institute of Technology, India. anikchand461@gmail.com
[3]Department of Information Technology, Haldia Institute of Technology, India.
bhowmickrudra07@gmail.com

Abstract

llmsays is an open-source Python library that reduces large language model (LLM) inference to a single function call. It introduces a two-stage hybrid routing mechanism that classifies an input prompt into one of four complexity tiers — `small`, `medium`, `large`, and `extra_large` — using a lightweight sentence-transformer model, and subsequently dispatches the query to the most appropriate provider-specific model. The library supports five commercial inference providers (Groq, NVIDIA NIM, OpenRouter, Fireworks AI, and Baseten) with automatic latency-aware failover, eliminating manual provider management for researchers and practitioners alike. An optional multiprocessing mode enables parallel provider queries, returning the first successful response for latency-critical applications.

Keywords: large language models, inference routing, provider failover, Python library, prompt complexity, sentence transformers, LLM abstraction.

Repository: https://github.com/abhirajadhikary06/llmsays **PyPI:** https://pypi.org/project/llmsays/ **License:** MIT

1. Introduction

Access to large language models has proliferated through numerous commercial API providers, each offering distinct models, pricing structures, rate limits, and failure modes. Developers who wish to query these services must typically author provider-specific client code, handle authentication, implement retry logic, and manually select models appropriate to a given task's complexity — a substantial engineering overhead that distracts from core research or application development goals.

llmsays addresses this friction by presenting a unified, single-function interface to five inference providers. Its core contribution is a hybrid prompt-tier router that automatically selects the minimal model capable of fulfilling a request, thereby balancing cost, latency, and capability. The entire interaction surface exposed to the end-user is:

```python
from llmsays import llmsays

response = llmsays("Explain quantum tunneling in simple terms")
print(response)
```

Listing 1: Minimal usage of `llmsays`.

Internally, the library determines prompt complexity, selects an appropriate model tier, dispatches the request to an available provider, and falls over gracefully to alternatives if a provider is unreachable or rate-limited.

2. Statement of Need

The contemporary LLM ecosystem presents several interoperability challenges:

- **API Fragmentation.** Each provider (OpenAI, Groq, NVIDIA NIM, etc.) exposes an incompatible authentication scheme, base URL, and SDK, necessitating provider-specific wrappers.
- **Model Selection Complexity.** Selecting a model commensurate with a task's complexity requires domain expertise; over-provisioning wastes compute budget, while under-provisioning degrades response quality.
- **Reliability.** Commercial APIs are subject to rate limits, transient outages, and quota exhaustion. Robust applications must implement retry and failover logic, which is non-trivial to do correctly.
- **Latency Variability.** Provider response times fluctuate. Applications requiring low latency benefit from parallelising requests across providers and accepting the first successful response.

Existing abstraction layers such as LiteLLM [2] provide broad model coverage but require explicit model selection by the caller. **llmsays** goes one step further by automating model selection through prompt-complexity routing, enabling fully hands-free LLM inference for standard use-cases.

3. Design and Implementation

3.1. Prompt-Tier Routing

The routing pipeline consists of two sequential stages designed for speed and accuracy:

Stage 1 — Heuristic Pre-filter. A lightweight rule-based filter examines surface-level features of the prompt (token count, presence of domain-specific keywords, structural complexity indicators such as multi-part questions or code snippets) to produce a coarse initial tier estimate at near-zero latency.

Stage 2 — Semantic Refinement. The prompt is encoded using `sentence-transformers/paraphrase-MiniLM-L3-v2` a 17M-parameter bi-encoder optimised for fast CPU inference. The resulting embedding is compared against centroid representations of each tier's exemplar prompts via cosine similarity. If the semantic signal conflicts with the heuristic estimate, the semantic score takes precedence.

The result is one of four tiers:

Tier	Intended Prompt Class
`small`	Simple factual queries, single-step tasks
`medium`	Multi-step reasoning, summarisation
`large`	Complex analysis, multi-document tasks
`extra_large`	Deep reasoning, large-context generation

3.2. Model Matrix

Each tier maps to a curated model per provider, as summarised in Table 1. The matrix is maintained in a versioned configuration file and can be extended by contributors as new models become available.

Table 1: Provider–model assignments per prompt tier (as of v1.x).

Provider	small	medium	large	extra_large
Groq	gpt-oss-20b	qwen3-32b	llama-3.3-70b	gpt-oss-120b
NIM	nemotron-nano-9b	llama-3.3-nemotron-49b	nemotron-120b	llama-3.1-nemotron-253b
OpenRouter	step-3.5-flash	gemini-3-flash	deepseek-v3.2	claude-opus-4.6
Fireworks	qwen3-8b	qwen3-vl-30b	qwen3-vl-235b	qwen3-coder-480b
Baseten	gpt-oss-120b	MiniMax-M2.5	Kimi-K2.5	GLM-5

3.3. Provider Failover and Latency Ordering

After tier selection, **llmsays** attempts providers in an order derived from an exponentially weighted moving average (EWMA) of observed response latencies, updated after each successful call within the process lifetime. If a provider raises a network error, authentication failure, or rate-limit exception, the library silently advances to the next candidate. This continues until a response is obtained or all configured providers are exhausted, in which case an informative exception is raised.

3.4. Parallel Query Mode

For latency-critical workloads, the optional `use_multiprocessing=True` flag submits the request to all configured providers simultaneously using Python's `concurrent.futures.ProcessPoolExecutor`. The

first successful response is returned and the remaining futures are cancelled. This trades additional API quota for reduced tail latency.

```
from llmsays import llmsays

# Explicit provider ordering with parallel execution
response = llmsays(
    "Design a production-ready distributed caching architecture",
    provider_preference=["Groq", "Openrouter", "fireworks-ai"],
    use_multiprocessing=True,
)
print(response)
```

Listing 2: Advanced usage: provider preference and parallel querying.

3.5. Command-Line Interface

llmsays ships with a CLI entry-point for interactive and scripted use:

```
# Basic query (auto-routed)
llmsays "Summarise the CAP theorem in three sentences"

# Restrict to specific providers
llmsays "Analyse this legal clause" --providers Groq Openrouter

# Parallel provider querying
llmsays "Generate a database schema for an e-commerce platform" \
    --use-multiprocessing
```

Listing 3: CLI examples.

3.6. Authentication

llmsays reads API credentials exclusively from environment variables, following the twelve-factor application convention [3]. At least one key must be present; supplying multiple keys enables failover:

`GROQ_API_KEY` `OPENROUTER_API_KEY` `NVIDIA_API_KEY` `FIREWORKSAI_API_KEY` `BASETEN_API_KEY`

4. Architecture Overview

The internal control flow of a single `llmsays()` invocation is as follows:

1. **Input validation** — Prompt is type-checked and stripped of extraneous whitespace.
2. **Heuristic pre-filter** — Coarse tier estimate produced from lexical features.
3. **Semantic routing** — MiniLM embedding compared against tier centroids; final tier assigned.
4. **Provider ordering** — EWMA latency scores determine provider priority queue.
5. **Dispatch loop** — Providers are attempted sequentially (or in parallel); first success returned.
6. **Latency update** — EWMA table updated for successful provider.

The library has a minimal dependency footprint: `sentence-transformers` for the embedding model, `httpx` for async-compatible HTTP, and standard-library modules for multiprocessing and environment variable handling.

5. Installation

llmsays requires Python ≥ 3.9 and is distributed via PyPI:

```
pip install llmsays
```

The `sentence-transformers` model weights are downloaded automatically on first invocation and cached locally via HuggingFace Hub.

6. Testing

The test suite is executed with `pytest` from the repository root. Tests cover the routing logic (unit tests with mocked embeddings), the failover mechanism (simulated provider errors), and CLI argument parsing. Contributions are expected to maintain or improve coverage.

7. Acknowledgements

The authors thank Haldia Institute of Technology for institutional support and the open-source community whose tooling — in particular the Hugging Face ecosystem and the `sentence-transformers` library — made this work possible.

References

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019. https://arxiv.org/abs/1908.10084

[2] BerriAI, "LiteLLM: Call all LLM APIs using the OpenAI format," 2023. https://github.com/BerriAI/litellm

[3] A. Wiggins, "The Twelve-Factor App," 2011. https://12factor.net/

[4] Groq Inc., "Groq LPU Inference Engine," 2024. https://groq.com/

[5] NVIDIA Corporation, "NVIDIA NIM Microservices," 2024. https://www.nvidia.com/en-us/ai/